

Lens — Problem framing

What problem does this product solve, and why does the problem exist in the first place?

This document answers the *Problem framing* and *Use case selection* items of the PwC Middle East "Design Your Own Production-Ready Full-Stack GenAI Solution" case study (sections 2 and 3.1 of the brief): the business problem and why it matters, the primary users and their roles, the core end-to-end journey, why GenAI is appropriate and where deterministic code remains necessary, the data being handled and its sensitivity, and the success criteria. The engineering detail behind each claim lives in [README.md](#) and [ARCHITECTURE.md](#).

1. The issue in the first place

Every operating business holds the answers to its everyday questions in a relational database — *how did revenue trend last quarter, which customers matter most, which carrier is late* — yet the people who ask those questions cannot get at them. Between the question and the answer sits a **request queue**:

1. A business user (a sales manager, a partner, a finance lead) has a question.
2. They cannot write SQL, and they are not allowed to connect to the database even if they could.
3. They raise a ticket with an analyst or the BI team.
4. The analyst reads the schema, writes and checks the query, builds a chart, and sends it back — typically **days later**, by which time the question has changed.
5. The chart is a screenshot or a spreadsheet: it goes stale immediately, cannot be re-run by the person who asked, and cannot be shared safely because it shows *the analyst's* data, not the recipient's.

The consequences are concrete:

Symptom	Cost to the organisation
Days of latency per question	Decisions are made on intuition or on last month's numbers
Analyst time spent on routine, one-off queries	Scarce technical people are a bottleneck for trivial work
Static, ad-hoc deliverables	The same question is asked and answered again every month
Screenshots forwarded by e-mail	Row-level access rules are bypassed the moment a chart leaves the analyst's screen
Long-tail questions never asked	Anything not already on a pre-built dashboard is invisible to the business

Traditional BI tools attack this with pre-built dashboards, which serve the *known* questions and leave the long tail unserved. Self-service BI attacks it with drag-and-drop query builders, which still require the user to understand the data model. Neither closes the gap for a non-technical user with a new question.

1.1 Why a plain "chat with your database" product is not enough

Large language models can now translate a plain-English question into SQL. That solves step 2 above, but on its own it creates three new problems that stop any organisation from putting such a

tool in front of real users on real data:

- **Safety.** A model that can write SQL can write `DELETE`, read a table it should not, or be talked into it by a user (prompt injection) — or by a value stored in the database itself.
- **Authorisation.** A model cannot be trusted to decide *which rows a user may see*. If the model is on the authorisation path, a wrong or manipulated answer leaks data across regions, business units or clients.
- **Persistence and cost.** A chat answer is ephemeral. If every look at a dashboard re-runs the model, the product is slow, expensive, non-deterministic (the same tile shows different numbers on different days) and stops working entirely when the provider is down.

So the issue is not merely "business users cannot write SQL". It is: **how do you let a non-technical user ask any question of governed data, in plain language, and keep the answer — without ever putting the model in charge of what is executed or who sees what.**

2. What Lens solves

Lens is a conversational analytics and dashboard composer. A business user types a question in plain English; a code agent writes a read-only SQL query against a curated, allow-listed set of views, executes it, and returns an answer with an interactive chart and the SQL that produced it. Any chart can be **pinned** to a dashboard, organised into groups, shared with colleagues, and **refreshed later with zero model calls**, always under the *viewer's* own data scope.

The design principle that resolves the three problems in §1.1 is:

Generate once, execute forever. *The model authors artefacts (SQL and chart code); deterministic code validates, executes and persists them. The model is never on the authorisation path and never required to re-render a dashboard.*

Mapped to the pain points:

Pain point	What Lens does about it
Days of latency	A question is answered in seconds, with the SQL visible for anyone who wants to check it
Analysts as a bottleneck	Routine questions are self-served; analysts curate the view catalogue and business rules instead of answering tickets
Static deliverables	A chart is a stored artefact (question + SQL + render code); it is refreshed on demand against live data
Leaked screenshots	A shared dashboard shows <i>each viewer</i> their own permitted rows — the scope is bound at execution time, by code, from the viewer's access record
Long-tail questions	Anything expressible over the curated views can be asked; the business rules (e.g. "shipped late", "days to ship") come from the catalogue, not from the model's guess
Model as a safety risk	Six fenced model calls; every statement is parsed, checked against the allow-list, capped and scope-bound before execution; blatant attacks are blocked before any model call
Model as an authorisation risk	Row-level scope is fetched, verified and bound by deterministic code; the model can <i>veto</i> a query, never widen it; absence of a scope record means no access

Pain point	What Lens does about it
Cost, latency and provider dependency of dashboards	Dashboard refresh re-executes stored SQL and stored chart code with <code>llm_calls == 0</code> , asserted in tests; a provider outage does not make the service unready

2.1 Business value

- **Time-to-answer** collapses from days to seconds for the long tail of questions that never make it onto a pre-built dashboard.
- **Analyst capacity** is redirected from one-off queries to curating the semantic surface (views, business rules, sensitive-column flags), which compounds: every rule added improves every future answer.
- **Governance improves rather than erodes.** Because sharing a dashboard shares the *question*, not the *data*, row-level access is enforced on every view of every tile — stricter than the e-mailed screenshot it replaces.
- **Predictable cost.** Model spend is incurred once per new question, never per dashboard view; per-user daily ceilings on turns, tokens and USD make the bill bounded and visible (`Usage & cost` screen, Prometheus metrics).
- **Auditability.** Every answer carries its SQL, prompt version, model, tokens, cost and a trace id; every guard block or repair is recorded. A regulated firm can show exactly what was executed, by whom, and why.

3. Primary users and their roles

User	Role in Lens	What they need	Demo account
Business user / decision maker (sales manager, regional lead, partner)	<code>viewer</code>	Ask questions in plain English; open dashboards shared with them; refresh tiles and see <i>their</i> data	<code>partner@lens.demo</code> — Eastern region only
Analyst / power user	<code>analyst</code>	Ask questions, pin charts, build and share dashboards, inspect the generated SQL, organise tiles into groups	<code>analyst@lens.demo</code> (all regions), <code>analyst2@lens.demo</code> (Eastern + Western)
Data / platform owner	<code>admin</code>	Curate the view allow-list and sensitive columns, manage users and scopes, monitor usage, cost, guard events and traces	<code>admin@lens.demo</code>

Roles are a **ceiling**; per-dashboard grants are the floor. A global `viewer` can never edit a dashboard, whatever grant they hold. A user without a grant receives `404`, never `403`, so the existence of dashboards they cannot see is never confirmed.

4. The core end-to-end journey

1. **Sign in** (local credentials in the PoC; OIDC Authorization Code + PKCE against Entra ID / Okta in production — see ARCHITECTURE §3.1).
2. **Ask** — "*Which shipper delivers fastest, and how often are orders shipped after the required date?*"
The turn runs as a 10-step pipeline: deterministic injection pre-screen → screen model → table

selection → agent writes SQL and chart code through two strict tools → SQL guard (parse, read-only, allow-list, sensitive columns, row cap, scope predicate) → read-only execution with a statement timeout → chart code runs in a forked, resource-limited sandbox → narrative phrased from findings → answer with chart, SQL panel and trace link.

6. **Pin** the chart. Pinning copies the already-rendered artefact — it never re-runs the model.
4. **Compose** a dashboard: add tiles, drag to arrange, optionally let the model *suggest* groups from tile titles (validated by code: every tile assigned exactly once, no invented ids, user confirms before anything is applied).
5. **Share** with a colleague as viewer or editor.
6. **Refresh** — by anyone with access, at any time. Each tile re-executes its stored SQL with *the refresher's* scope bound in, and re-runs its stored render code. `0 model calls`, visible on the tile and in the trace.
7. **Operate** — an admin watches usage and cost per user/day, guard blocks by reason, stage latencies, provider health and the shadow-verdict disagreement rate that decides when the deterministic parser can replace the guard model.

5. Why GenAI is appropriate — and where deterministic code must remain

5.1 Where GenAI adds clear value

The problem is fundamentally one of **translation between an open-ended natural-language question and a formal query over a schema the user does not know**. That is exactly the task LLMs are good at and that deterministic code cannot do: the space of questions is unbounded, the mapping from business vocabulary to columns is fuzzy, and the output (SQL + chart) is structured code that can be *checked* by a machine before it is trusted. Lens uses a model in six places, each producing structured, schema-validated output:

Stage	Model output	Why a model, not code
Injection screen	<code>allow/block</code> + category (strict JSON schema)	Subtle social-engineering phrasing is not regex-shaped; a deterministic pre-screen catches the blatant cases first, at zero cost
Table selection	list of relevant views	Keeps the schema prompt small as the catalogue grows; fails <i>open</i> to the full schema — a cost optimisation, never a security control
Agent	SQL and Python chart code via two tool calls	The core translation task
Guard verifier	verdict on scope-predicate <i>placement</i> across joins, CTEs, subqueries	The subtle semantic question; the parser records a shadow verdict so promotion to code is data-driven
Narrative	findings phrased from chart titles	Natural-language summarisation
Grouping	proposed dashboard sections from tile titles	Semantic clustering of free text

5.2 Where deterministic code is non-negotiable

Anything that decides **what runs** or **who sees what** is code, tested against the real components:

- **Authentication and authorisation** — FastAPI dependencies; role read from the database row on every request, not from the token.
- **Row-level scope** — fetched from `access_scope`, the predicate `region_id IN (:lens_scope_region_id)` is *verified and repaired* in the SQL AST by `sqlglot` and *bound* to the caller's literals immediately before execution. The verifier model can veto; deterministic repair always runs afterwards and only ever narrows. Fail-closed on every path.
- **SQL guard** — single statement, `SELECT` only, allow-listed views only, no system catalogues, no sensitive columns, row cap, statement timeout. 51 hostile statements blocked, 0 false positives.
- **Execution** — the analytics file is opened read-only with an authorizer callback that refuses base tables and unlisted views at the database layer, independently of the guard.
- **Chart rendering** — generated Python runs in a forked child with memory/CPU/file/process limits, no network, no database handle, restricted imports and builtins.
- **Persistence, refresh, grouping apply, completeness checks** — plain code, single transactions.
- **Grounding** — an answer with no query behind it is refused (`ungrounded_answer`); the model may not invent figures. Instruction-like text arriving *from data* is redacted, never obeyed.

The rule of thumb used throughout: **the model proposes, code disposes**. Every model output is a structured artefact that deterministic code validates before anything is executed, stored or shown.

6. The data being handled and its sensitivity

Data	Where it lives	Sensitivity	Handling
Business analytics data — Northwind Traders: 16,282 orders, 609,283 order lines, customers, products, suppliers, employees, shippers (Jul 2012 – Oct 2023)	SQLite file, opened <code>mode=ro</code> , reached only through ten curated <code>v_*</code> views	Commercially confidential; partitioned by sales region as the row-level scope key	Users see only the rows of the regions in their access record; absence of a record means no access
Personal data inside that dataset — customer and supplier contact names and phone numbers	Columns flagged <code>sensitive</code> in the catalogue	Personal data	Never rendered into the prompt, refused if referenced in SQL, <code>SELECT *</code> on views carrying them is refused, redacted from logs
Lens's own state — users, Argon2id password hashes, rotating refresh tokens, dashboards, charts (question + SQL + render code), traces, usage counters, audit log	PostgreSQL, separate engine and credentials, no cross-database joins	Credentials and audit data are sensitive; charts are as sensitive as the questions asked	Short-lived in-memory access tokens; <code>HttpOnly</code> ; <code>SameSite=Strict</code> refresh cookie with reuse detection; trace retention window
What reaches the model provider	Outbound API calls only	Schema metadata (names, types, descriptions), business rules, the user's question, aggregated query results as DataFrame summaries, chart titles	Row samples off by default; sensitive columns excluded from schema context; no credentials or DSNs; prompt id and content hash recorded on every call

Data	Where it lives	Sensitivity	Handling
Secrets — provider API key, database credentials, JWT keys	Environment variables only	Highest	<code>.env</code> git-ignored, <code>.env.example</code> carries no secrets, CI runs gitleaks, values masked in logs

In the illustrative Middle East professional-services setting this stands in for — client engagement data, regional P&L, partner-level views — the same mechanism applies: the scope key becomes *client*, *engagement* or *business unit*, and a partner sharing a dashboard with a manager shares the question, never the rows.

7. Success criteria

Criterion	Target	Evidence in the repository
A non-technical user gets a correct, charted answer from a plain-English question	≥ 90 % of golden-set questions runnable, ≥ 85 % matching reference	20-question golden set: 20/20 runnable, 18/20 matching against <code>gpt-4o</code> ; hermetic half runs in CI
No model on the authorisation path	0 statements executed without a bound scope predicate	<code>test_sql_guard.py::test_zero_queries_reach_executor_without_verified_scope</code>
Dashboards work without the model	<code>llm_calls == 0</code> on every refresh; service stays ready when the provider is down	<code>test_refresh.py</code> , <code>test_ops.py</code> , <code>/health/ready</code> reports provider as informational
Hostile input is contained	Injection, DDL/DML, catalogue reads, base tables, poisoned data values all blocked or neutralised	51-statement adversarial corpus, <code>test_poisoned_row_is_data_not_instruction</code> , read-only DB tests that bypass the guard on purpose

Criterion	Target	Evidence in the repository
Sharing never widens access	A narrower viewer sees fewer rows on the same tile; viewer can never edit	<code>test_refresh.py</code> , <code>test_dashboards.py</code> (404 not 403, role ceiling)
Cost is bounded and visible	Per-user daily ceilings on turns / tokens / USD; cost per turn recorded	<code>usage_counter</code> , Usage & cost screen, Prometheus <code>/metrics</code>
Interactive latency	≤ 12 s p95 per turn	≈ 10 s measured against OpenAI directly; 15–25 s on a free gateway — stated as a known gap
222 backend tests green against real guard, sandbox, auth and databases	All pass	<code>pytest -q</code>

8. Scope chosen, and why

The brief asks for depth over breadth. Lens deliberately narrows to **one governed data source, one complete journey (ask → pin → compose → share → refresh), and the hard parts of making that safe**: row-level scope enforced by code, a SQL guard with an adversarial corpus, a sandboxed executor, zero-model refresh, and full observability. Enterprise SSO, container-per-execution sandboxing, database-level RLS, scheduled refresh, a semantic/metric layer, cross-filtering and streaming are designed, documented with their intended production shape, and explicitly deferred (ARCHITECTURE §5), because none of them demonstrates anything new about the problem being solved.